QUIZ MASTER – V2 PROJECT REPORT

1. Student Details

- Name: Chinmay Kashyap C S
- Roll Number: 23F2001372
- Course: Modern Application Development (MAD2)
- Project Title: Quiz Master – V2

2. Project Details

2.1 Problem Statement

The objective of this project is to develop a multi-user exam preparation platform where:

- An Admin (Quiz Master) manages subjects, chapters, quizzes, and users
- Users can register, attempt quizzes, and track their performance

The system must include:

- Role-based authentication
- Quiz management
- Score tracking
- Background jobs (reminders, reports, exports)
- Performance optimization using caching

This project follows all constraints such as:

- SQLite database only
- Flask for backend APIs
- Vue.js for frontend
- Redis + Celery for caching and background jobs

2.2 Approach to the Problem

The problem was approached using a full-stack modular architecture:

Step 1: Backend Design

- Designed REST APIs using Flask
- Implemented JWT-based authentication

- Created role-based access control (Admin/User)

Step 2: Database Modeling

- Designed relational schema:
    - User → Subject → Chapter → Quiz → Question → Score
- Implemented using SQLAlchemy ORM

Step 3: Frontend Development

- Built UI using Vue.js + Bootstrap
- Created dashboards for Admin and Users
- Integrated APIs with frontend

Step 4: Background Jobs

- Used Celery for async tasks
- Implemented:
    - Daily reminders
    - Monthly reports
    - CSV exports

Step 5: Performance Optimization

- Used Redis caching
- Added cache invalidation after updates

3. Frameworks and Libraries Used

Backend

- Flask (REST API framework)
- Flask-RESTful
- Flask-JWT-Extended (authentication)
- SQLAlchemy (ORM)

Frontend

- Vue.js
- Vue Router
- Bootstrap

Database

- SQLite

Caching & Async

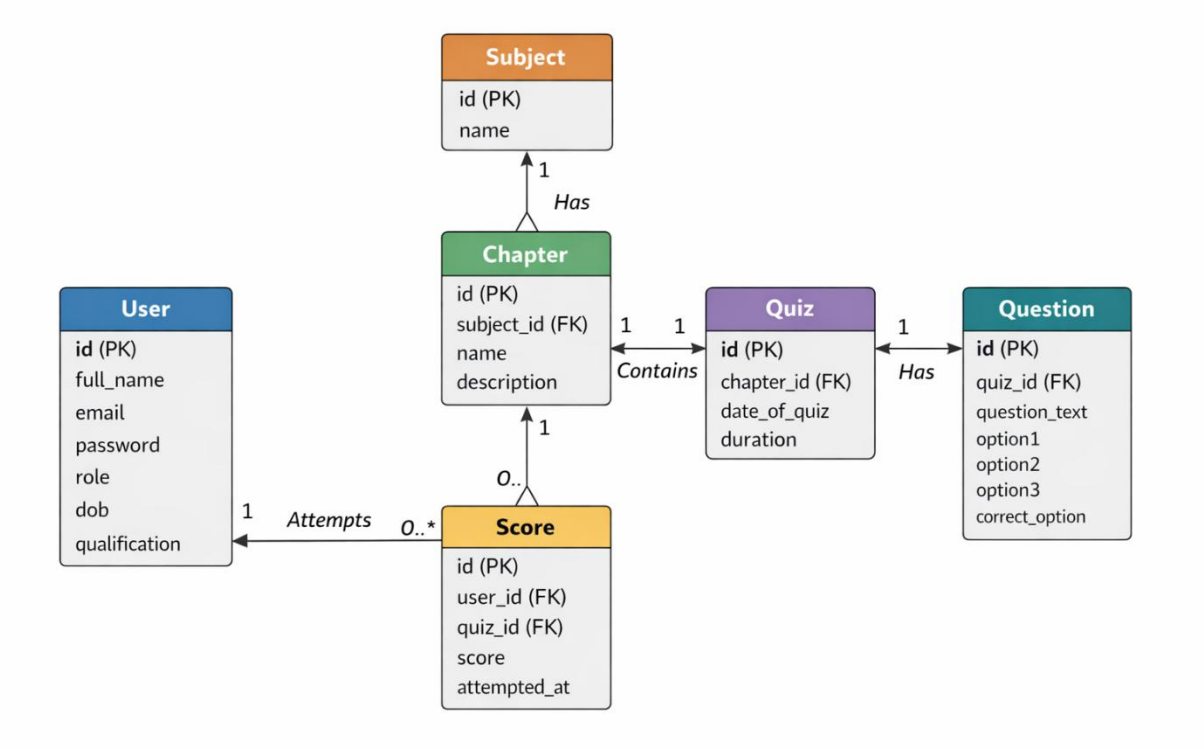
- Redis (caching)
- Celery (background jobs)

Other Libraries

- ReportLab (PDF generation)
- Matplotlib (graph generation)
- SMTP (email service)

4. ER Diagram (Database Design)

Entities and Relationships



Relationship Summary

- One User → Many Scores
- One Subject → Many Chapters
- One Chapter → Many Quizzes
- One Quiz → Many Questions

This ensures structured and normalized database design.

5. API Resource Endpoints

Authentication APIs

- POST /api/register → Register user
- POST /api/login → Login
- PATCH /api/editprofile → Update profile

Admin APIs

- GET /api/admin/users → View users
- POST /api/subjects → Create subject
- PATCH /api/subjects/<id> → Update subject
- DELETE /api/subjects/<id> → Delete subject

Chapter APIs

- GET /api/chapters
- POST /api/chapters
- PATCH /api/chapters/<id>
- DELETE /api/chapters/<id>

Quiz APIs

- GET /api/quizzes
- POST /api/quizzes
- PATCH /api/quizzes/<id>
- DELETE /api/quizzes/<id>

Question APIs

- GET /api/questions
- POST /api/questions
- PATCH /api/questions/<id>
- DELETE /api/questions/<id>

Quiz Attempt APIs

- GET /api/quiz/attempt/<quiz_id> → Start quiz
- POST /api/quiz/attempt/<quiz_id> → Submit quiz

Other APIs

- GET /api/scores → View scores
- POST /api/export/csv → Export user CSV
- POST /api/admin/export/csv → Export admin CSV
- GET /api/reports → Download reports

(Implemented as per backend code)

6. Presentation Video

Drive: *https://drive.google.com/file/d/197RT3wddbNDZ7qB9NQLd-cgXEEzm4EIB/view?usp=sharing*

7. Conclusion

The project successfully implements a scalable, full-stack quiz platform with:

- Clean architecture
- Proper database design
- Efficient API structure
- Background job processing
- Performance optimization

It demonstrates strong understanding of:

- Web development
- System design
- Asynchronous processing